LagPilot: Learning When to Scale Kafka Consumers with Q-learning

CSCI3235 Reinforcement Learning, Midterm I

Aliyev Nurislam, Student ID: 24B030024, 26 October 2026

1. Problem and RL suitability

Task. An online shop writes every order to a Kafka topic `orders` with 4 partitions. A consumer group reads and processes the orders. The decision-maker is an autoscaler. Every 10 seconds it chooses to keep the consumer count, add one consumer, or remove one (1 to 4 consumers). Traffic follows a daily curve (20 RPS at night, 120 RPS at the peak) plus random flash-sale spikes that double the traffic. Too few consumers make the consumer lag grow, and orders break the SLA: "an order must be processed within 30 seconds". Too many consumers waste money. Every change of the consumer count triggers a *rebalance*. During a rebalance the whole group stops consuming for 20 seconds (eager, stop-the-world rebalance). The desired outcome is low delay, few SLA violations and few consumers, with no more rebalances than needed.

Why decisions are sequential. Lag accumulates: messages not processed now are still waiting in the next step. A rebalance started now costs throughput in the *next* steps. Traffic changes over time, so a consumer added before a spike pays off later. The agent must learn from experience how much a rebalance really costs and when scaling is worth it, because the traffic distribution is not given to it.

Non-RL alternative. (1) The KEDA rule used in Kubernetes: `desired = ceil(lag / lagThreshold)`. (2) Capacity planning: estimate peak RPS and set the consumer count from Little's Law. These alternatives are better when traffic is stable or predictable, when rebalances are cheap (for example cooperative incremental rebalancing), and when the operations team needs a rule that is easy to explain and audit. RL may be useful when rebalances are expensive and traffic is bursty, because a fixed rule cannot weigh the future cost of a rebalance.

Capacity estimate. One consumer processes $\mu = 50$ msg/s. At the peak, 120 RPS / 50 = 2.4, so at least 3 consumers are needed. At night, 20 RPS needs only 1 consumer. A spike at the peak gives 240 RPS. That is more than the maximum capacity of $4 \times 50 = 200$ msg/s, so lag *must* grow by about 40 msg/s (400 messages per step). No policy can avoid this. The policy can only manage the lag.

Application link. I work as a backend developer; my *Backend for Highloaded Environment* course covers Kafka, consumer groups, capacity estimation, Little's Law, SLAs and autoscaling. This project is a first step toward a cost-aware consumer autoscaler.

2. MDP formulation

Time and horizon. One step is 10 s. One episode is H = 120 steps (20 minutes, a compressed "day"). The task is **episodic** with a fixed horizon.

Start state. 2 consumers, lag = 0, no rebalance.

Actions (3). `0 keep`, `1 scale_up` (+1, max 4), `2 scale_down` (−1, min 1). An invalid action (scale_up at 4, scale_down at 1) acts as `keep`. Actions during a rebalance are ignored and act as `keep`. More than 4 consumers is useless, because one partition is read by at most one consumer.

Transitions (stochastic). Each step: (1) the action may start a rebalance of R = 2 steps; the new count applies when it ends. (2) Arrivals ~ Poisson(RPS × 10), where RPS = base curve × 2.0 during a spike; a spike starts with probability 0.04 per step and lasts 5–15 steps. (3) Processed = min(L + arrivals, $c \times \mu \times 10$), or 0 during a rebalance. (4) Lag L ⟵ L + arrivals − processed. (5) Delay by Little's Law: `delay_s = L / (c × μ)`. Little's Law says waiting time = items in the system / throughput; here the items are the lag and the throughput is the current capacity $c \times \mu$.

State (288 discrete states). delay_level (4: < 5, < 15, < 30, ≥ 30 s) × lag_trend (3: falling / stable / rising, with a dead-band of max(5%, 100 msgs)) × rps_level (3: < 40, < 90, ≥ 90 RPS measured in the last step, like the broker metric `MessagesInPerSec`) × consumers (4) × rebalancing (2). Without rps_level there are 96 states (used in Section 5.5).

Reward (negative cost per step).

$$r = -\left(1.0 \cdot \frac{\min(\text{delay_s}, 120)}{10} + 2.0 \cdot c + 5.0 \cdot \mathbb{1}[\text{delay_s} > 30] \right)$$

The first term punishes waiting orders, the second pays for servers, the third punishes SLA violations. The cap at 120 s stops one bad step from dominating the return. A rebalance has no direct cost; its cost appears through the lag it creates. This is exactly what the agent must learn.

Discount. γ = 0.95. The effective horizon is about 1/(1−γ) = 20 steps (200 s). This is long enough to see the cost of a 20 s rebalance and the end of a typical spike (5–15 steps), and short enough to learn quickly. At the last step (t = H) we use target = r, so we treat the time limit as terminal.

Stochasticity. The environment is stochastic (Poisson arrivals, random spikes). The behaviour policy during training is ε-greedy, so it is stochastic. The evaluated policy is greedy and deterministic.

Markov property. The state is only *approximately* Markov. The agent does not see: the exact lag (only a delay bucket), the true RPS (only its level), whether a spike is running and how long it will last, the position in the daily curve (no time-of-day feature), and how many rebalance steps are left. The same discrete state can therefore lead to very different futures. Adding rps_level gives the agent part of the missing information; Section 5.5 tests what this changes.

Transition example (real step from `results/transition_example.json`, evaluation seed 1000, step 0, Q-learning seed 0):

	delay_level	lag_trend	rps_level	consumers	rebalancing
state s (id 26)	low	stable	low	2	no
action a	**scale_down**				
next state s′ (id 51)	low	rising	low	2	**yes**

Raw numbers: RPS = 20.0, arrivals = 196, processed = 0 (the rebalance stopped the group), lag = 196, delay = 196 / (2 × 50) = 1.96 s, no SLA violation. Reward = −(1.96/10 + 2.0 × 2 + 0) = **−4.196**. Q-values in s: keep −81.71, scale_up −79.37, scale_down −76.63, so the greedy action is scale_down: at night one consumer is enough, and the agent pays one short rebalance to save one server for the next minutes.

3. Method: tabular Q-learning

We use Q-learning (Sutton & Barto, 2018, Section 6.5). The agent keeps a table Q(s, a) of size 288 × 3, started at zeros. Q(s, a) estimates the **expected discounted return** if we take action a in state s and then act greedily. After each step:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- **Q(s, a)**: our current guess of how good action a is in state s.
- **r**: the reward we just got (minus the cost of this step).
- **γ max_a′ Q(s′, a′)**: the value of the best action in the next state, discounted by γ = 0.95. This is the "future" part. Q-learning uses the *best* next action, not the action we will really take, so it is **off-policy**: it learns the greedy policy while exploring.
- **r + γ max Q(s′, ·)**: the TD target, a better guess built from one real step.
- **target − Q(s, a)**: the TD error, how wrong the old guess was.
- **α = 0.1**: the learning rate, how far we move toward the target. A small α averages over the random arrivals and spikes.
- At the last step of the episode the target is just r.

How the policy uses the estimate. During training the agent uses ε-greedy: with probability ε it picks a random action (exploration), otherwise it picks argmax_a Q(s, a) (exploitation). ε decays linearly from 1.0 to 0.05 over the first 70% of the episodes. During evaluation the agent always picks argmax_a Q(s, a) (ε = 0, ties go to `keep`) and does not learn.

Why Q-learning. The model is small and tabular. Q-learning updates every step (unlike Monte Carlo), which helps in long episodes of 120 steps, and it learns the greedy policy directly. SARSA would also fit; we chose Q-learning because the evaluation policy is greedy.

Baselines. (1) **KEDA rule**: `desired = clip(ceil(L / 500), 1, 4)`, then one step toward desired. It reads the exact lag but ignores rebalance cost and RPS. The threshold 500 was the best of {250, 500, 1000, 2000} for KEDA itself. (2) **Static max**: scale to 4 consumers and keep them. (3) **Random**: uniform random actions.

4. Experimental setup

- **Training:** 10 independent seeds (0–9), 3000 episodes per seed. Each training episode has its own traffic seed, separate from all evaluation seeds.
- **Learning curve:** every 50 episodes, a greedy check (no learning) on 10 fixed check seeds (2000–2009).
- **Final evaluation:** each trained agent runs greedily for 50 episodes on evaluation seeds 1000–1049, which were never used for training or tuning. Baselines run on the same 50 seeds. We report mean ± std over episodes (Q-learning: 10 seeds × 50 = 500 episodes) and the std of the 10 per-seed means.

- **Config:** frozen values in the README table (main values: μ = 50 msg/s, R = 2 steps, spike × 2.0, w = (1, 2, 5), α = 0.1, γ = 0.95). The config was tuned once, with baselines only, on seeds 5000–5039, *before* any Q-learning run, so that 1 consumer fails at the peak, 4 consumers are enough but expensive, and spikes hurt.
- **Training length.** Figure 1 shows that the mean curve rises until ε reaches 0.05 (episode 2100) and then stays in the same band instead of improving further, so 3000 episodes (about 900 episodes with ε = 0.05 after the decay) is enough for this table size.
- **Reproducibility:** `python -m lagpilot.experiments` re-creates every figure and CSV in about 2 minutes. All numbers below come from `results/*.csv`.

5. Results

5.1 Training curve

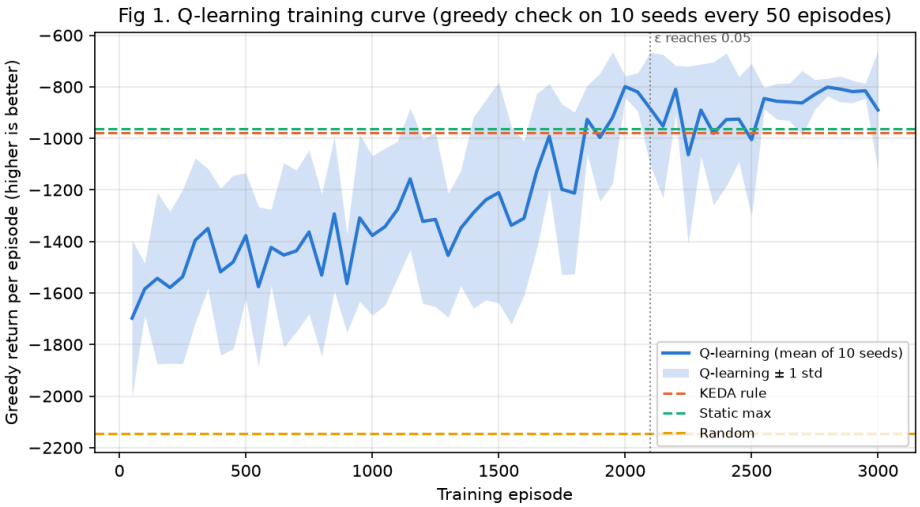


Figure 1: Greedy check return during training, mean ± std over 10 seeds. Dashed lines: baselines on the same check seeds.

The greedy return improves from about −1698 (episode 50) to about −800 (episode 2000, std 41.8 across seeds). While ε is large, the curve is noisy. After ε reaches 0.05 the mean stays mostly above both KEDA and static max, but it is **not stable**: at some checkpoints the mean drops (for example −1005.2 at episode 2500) and the std becomes large again (228.2 at episode 3000). This is caused by single seeds whose greedy policy changes after a few updates (Section 6).

5.2 Final comparison

Policy	Return	Delay (s)	SLA viol. %	Consumers	Rebalances
Q-learning	**−897.4 ± 264.2**	30.1 ± 84.0	14.7 ± 14.6	**2.68 ± 0.28**	10.5 ± 6.1
KEDA rule	−1002.6 ± 84.9	15.3 ± 2.3	10.5 ± 7.1	3.15 ± 0.18	26.6 ± 4.0
Static max	−961.5 ± 8.7	**0.8 ± 0.7**	**0.0 ± 0.0**	3.97 ± 0.00	2.0 ± 0.0
Random	−2126.7 ± 129.2	332.9 ± 114.6	78.3 ± 8.4	2.51 ± 0.35	40.7 ± 3.2

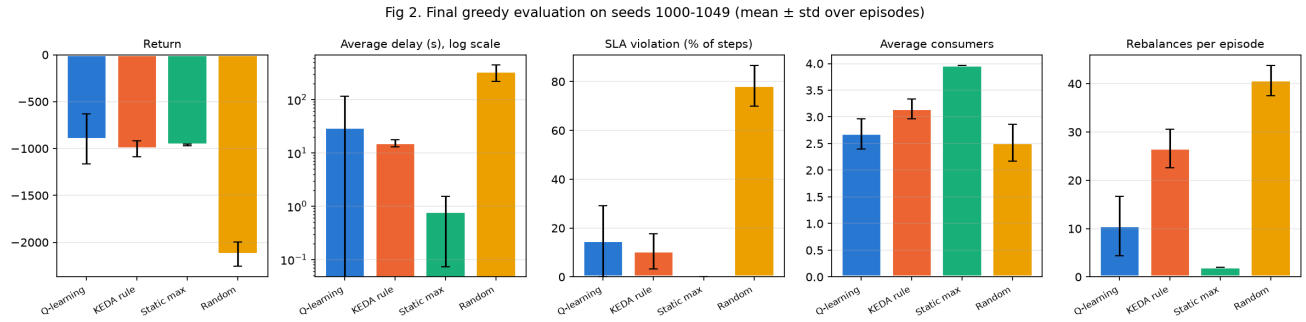


Figure 2: Final greedy evaluation on seeds 1000–1049 (mean ± std over episodes).

On average Q-learning has the best return. It uses fewer consumers than KEDA (2.68 vs 3.15) and makes 2.5 times fewer rebalances (10.5 vs 26.6). But it has *more* SLA violations than KEDA (14.7% vs 10.5%): it accepts some delay to save servers, which is what the reward weights ask for. Static max has almost no delay but pays for 4 servers all the time. The large

Q-learning std comes mostly from **one failed seed**: per-seed mean returns (`table_q_per_seed.csv`) are between −874.8 and −772.7 for 9 seeds, but −1537.8 for seed 2 (average delay 219.6 s, 48.5% SLA violations). The median of the 10 per-seed means is −829.7. So 9 of 10 agents beat both baselines; one agent is worse than both.

5.3 Parameter study: rebalance duration R

Fig 3. Effect of rebalance duration (Q-learning retrained for each R, 10 seeds)

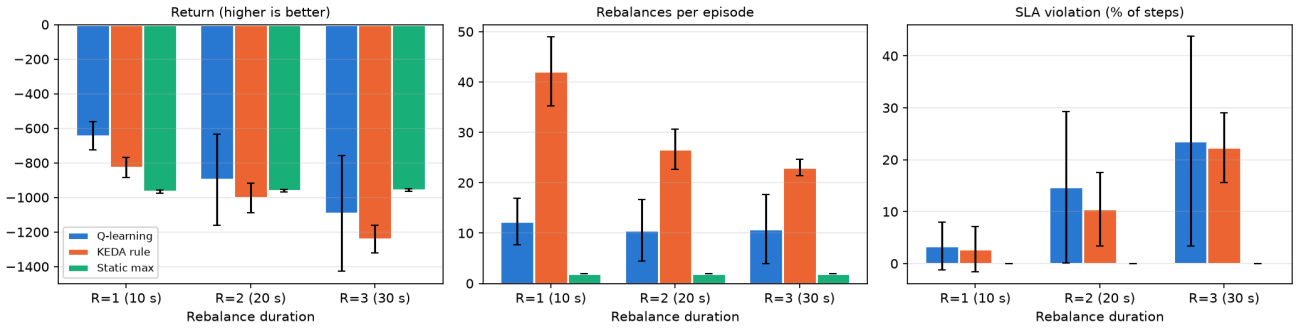


Figure 3: Q-learning retrained for each R (10 seeds), compared with KEDA and static max.

R	Q return	KEDA	Static	Q reb.	KEDA reb.	Q seeds > KEDA / > static
1 (10 s)	−643.4 ± 82.3	−826.9	−966.5	12.3	42.1	10 / 10
2 (20 s)	−897.4 ± 264.2	−1002.6	−961.5	10.5	26.6	9 / 9
3 (30 s)	−1092.7 ± 333.4	−1239.7	−957.3	10.8	23.1	8 / 4

(Seed counts from `fig3_q_per_seed.csv`.) We expected that with more expensive rebalances the agent would scale less often and the gap to KEDA would grow. **This only partly happened.** KEDA gets much worse as R grows (−826.9 ⟶ −1239.7) because it keeps flapping (23–42 rebalances). Q-learning makes far fewer rebalances than KEDA at every R, and its mean beats KEDA at every R (gap 183.4, 105.2, 147.1), but the gap does **not** grow steadily, and the number of Q-learning rebalances hardly changes (12.3 ⟶ 10.8). Most important: at R = 3, **static max is the best policy on average** (−957.3 vs −1092.7), and only 4 of 10 Q-learning seeds beat it. When a rebalance stops the group for 30 s, never scaling at all is a strong, simple choice, and our agent does not learn this reliably within 3000 episodes. With cheap rebalances (R = 1) Q-learning wins clearly and stably (std of the return 82.3).

5.4 One episode in detail

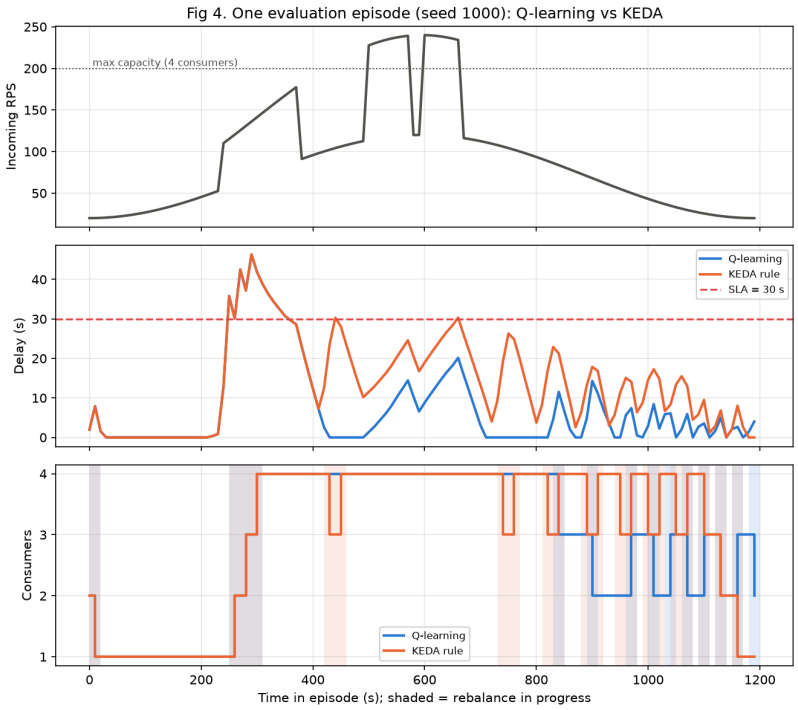


Figure 4: Evaluation seed 1000: incoming RPS, delay with the SLA line, consumer count (shaded = rebalance).

In this episode (`fig4_episode_trace.csv`) both policies act the same until 420 s: scale down to 1 at night, then scale up to 4 when the first spike arrives (250–290 s). Neither can prevent the SLA violation during that spike. After that, KEDA flaps: every time lag drops it removes a consumer, the rebalance stops consumption, lag rises, and it adds the consumer back (at 420/440 s, 730/750 s, and then every 50–70 s in the evening). Q-learning keeps 4 consumers through the whole busy part until 830 s, so its delay stays lower. In the evening it also switches between 2 and 3 consumers several times, so it is not free of flapping either. Totals for this episode: Q-learning 14 rebalances, 11 SLA-violating steps, return −833.5; KEDA 21 rebalances, 13 SLA-violating steps, return −954.2.

5.5 Does observing the RPS level help? (Markov study)

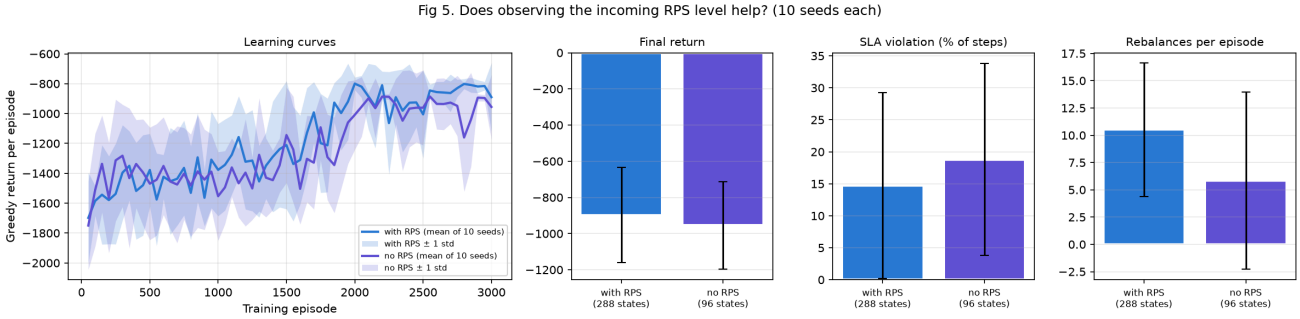


Figure 5: Q-learning with rps_level (288 states) vs without it (96 states), 10 seeds each.

With rps_level (288 states) the return is −897.4 ± 264.2 (median of seed means −829.7); without it (96 states) it is −953.5 ± 241.8 (median −876.1). Without the RPS level, the agent only sees the *consequence* of traffic (delay and lag trend), not its cause. It scales later, so it has more SLA violations (18.8% vs 14.7%). It also learns a more passive policy with fewer rebalances (5.8). With rps_level, the median agent is about 46 return points better and can scale up when RPS is high *before* the delay becomes critical. Each variant has one failed seed (seed 2 with RPS, seed 4 without), so the means are close; the improvement is clearer in the medians (`fig5_q_per_seed.csv`). Adding information to the state helps, but the larger state (288 vs 96) also needs more data.

6. Discussion, limitations and next steps

Failure: unstable seeds. One of 10 seeds (seed 2) learned a bad greedy policy (−1537.8), and the learning curve can drop after $\varepsilon = 0.05$. The policy heatmap (`fig6_policy_heatmap.png`) also shows strange choices, for example *scale_down* when delay is critical and RPS is medium (consumers = 2). Such states are visited rarely, so their Q-values rest on few noisy

samples, and the greedy choice can flip after a few updates. With $\alpha = 0.1$ and very noisy returns (spikes), small changes in Q can change the argmax. Tabular Q-learning gives no guarantee for rarely visited states within 3000 episodes.

Limitations. (1) *Discretization*: four delay buckets lose information; 31 s and 300 s look the same. (2) *No time-of-day feature*: the agent cannot know that night is coming; this breaks the Markov property. (3) *Eager rebalance model*: modern Kafka supports cooperative incremental rebalancing (KIP-429), where only moved partitions stop; this would make rebalances much cheaper and make KEDA stronger. (4) *KEDA without stabilization*: real KEDA/HPA uses cooldown and stabilization windows that reduce flapping; our rule has none, so the comparison is favourable to Q-learning. (5) *Simulator gap*: real consumers have variable throughput, rebalances have random length, and traffic is not a clean sine curve.

What I would test next. Lower α or decaying α and more episodes, to fix the unstable seeds; select the final policy by validation return instead of using the last Q-table; add a time-of-day feature and a "spike active" feature; compare with SARSA; add a KEDA rule with a cooldown window; model cooperative rebalancing (shorter R); and run the policy on a real Kafka cluster (the demo in `demo/` is a first step, but it is not part of the results).

7. AI Use Statement

Tool: Claude Code (Anthropic, model Claude Opus 5.5). **Used for:** generating the first version of the simulator, the agents, the experiment script and the figures from my detailed specification; proposing the one-time config tuning (checked only on baselines); drafting the README and this report from the CSV results; drafting the optional demo code. **What I verified myself:** I ran the live Kafka demo myself (API, load generator and agent) and checked in the agent log that the trained policy added a consumer when the incoming RPS grew and that a real Kafka rebalance followed.

8. References

1. R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. (Ch. 3: MDPs; Ch. 6.5: Q-learning.) http://incompleteideas.net/book/the-book-2nd.html
2. Apache Kafka Documentation: consumer groups, consumer configuration and group rebalance. https://kafka.apache.org/documentation/
3. KIP-429: Kafka Consumer Incremental Rebalance Protocol. https://cwiki.apache.org/confluence/display/KAFKA/KIP-429%3A+Kafka+Consumer+Incremental+Rebalance+Protocol
4. KEDA documentation: Apache Kafka scaler (`lagThreshold`). https://keda.sh/docs/latest/scalers/apache-kafka/
5. J. D. C. Little. A proof for the queuing formula $L = \lambda W$. *Operations Research*, 9(3), 1961.
6. CSCI3235 Midterm I Take Home Project description, KBTU, 2026.